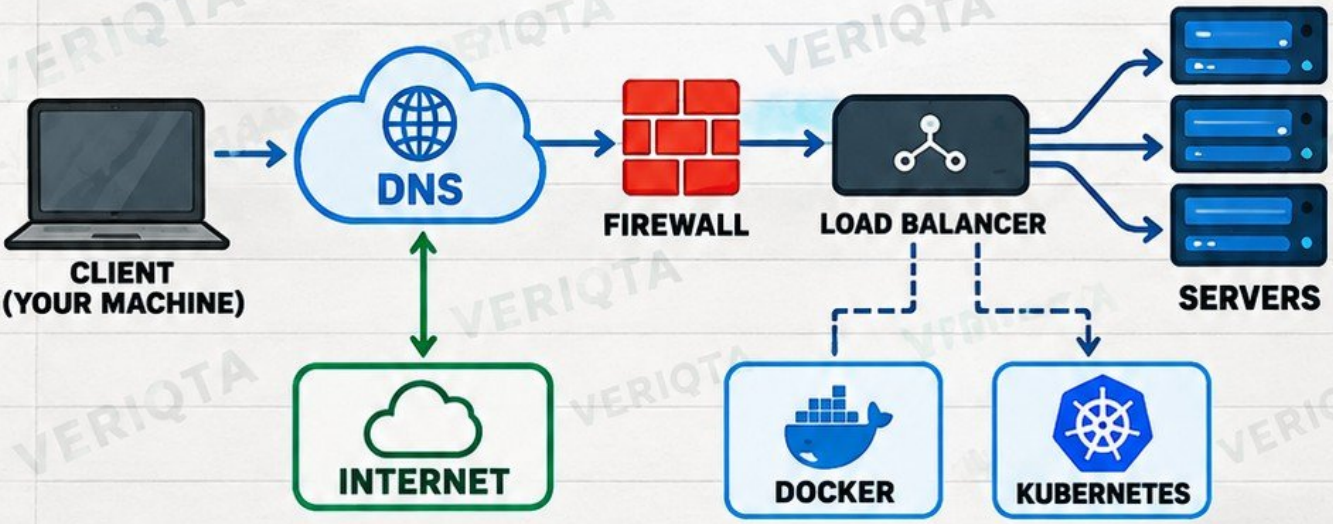
BEGINNER FRIENDLY

NETWORKING LAB MANUAL FOR DEVOPS

PRACTICAL NETWORKING SKILLS FOR LINUX, DOCKER, CLOUD AND KUBERNETES



- HANDS-ON LABS
- REAL-WORLD SCENARIOS
- TROUBLESHOOTING GUIDES
- LINUX DOCKER CLOUD
- SECURITY BEST PRACTICES
- PRODUCTION INSIGHTS

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

VERIQTA

PAGE 01

VERIQTA

Follow One Request from Your Laptop to a Web Server

Trace what happens when you run curl https://example.com. Identify DNS resolution, TCP connection, TLS negotiation, HTTP response, and where each stage can fail.



YOUR LAPTOP (Client) → **DNS** Finds the IP address → **TCP** Builds a connection → **TLS** Encrypts the traffic → **HTTP** Returns the response → **WEB SERVER** (example.com)

1 DNS RESOLUTION

- Converts the domain name to an IP address.
- Uses your system DNS resolver.

2 TCP CONNECTION

- Client opens a TCP connection to the server IP on port 443 (for HTTPS).
- Performs a three-way handshake.

3 TLS NEGOTIATION

- Client and server agree on encryption settings.
- Server presents a certificate.
- Client verifies it.

4 HTTP REQUEST

- Client sends an HTTP request (GET /).
- Includes headers such as Host, User-Agent, etc.

5 HTTP RESPONSE

- Server returns a response (e.g. 200 OK).
- Browser or curl displays the content.

1. CHECK DNS RESOLUTION

```
getent ahosts example.com
```

- Shows the IP address (or addresses) returned by your DNS resolver.
- Use this to confirm DNS is working and which IP will be used.

2. CHECK THE NETWORK ROUTE

```
ip route get <server-ip>
```

- Shows the route your machine will use to reach the server IP.
- Helps you verify the correct network interface and gateway.

3. MAKE A VERBOSE REQUEST

```
curl -v https://example.com
```

- Shows each step: DNS, TCP, TLS and HTTP request/response.
- Useful for seeing exactly where a connection fails.

EXAMPLE OUTPUT (PARTIAL)

```
*   Trying 93.184.216.34:443...
*   Connected to example.com (93.184.216.34) port 443
*   TLS handshake, Client hello (1):
*   TLS handshake, Server hello (2):
*   SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384
>   GET / HTTP/1.1
>   Host: example.com
<   HTTP/1.1 200 OK
<   Content-Type: text/html
<   Content-Length: 1256
...
```

WHERE IT CAN FAIL

1. **DNS failure** – Cannot resolve the domain name.
2. **TCP failure** – Connection refused, timeout, or unreachable.
3. **TLS failure** – Certificate issues, handshake failure, unsupported version or cipher.
4. **HTTP failure** – 4xx or 5xx errors from the server.
5. **Network filtering** – Firewall or security group blocks traffic.

TRY THIS (LAB)

1. Run: `getent ahosts example.com` and note the IP address.
2. Run: `ip route get <server-ip>` and check the route.
3. Run: `curl -v https://example.com` and observe each stage.
4. Try a domain that fails DNS (e.g. `curl -v https://nonexistent12345.com`) and compare the error.

JUNIOR ENGINEER TIP

- Always test each layer separately.
- Do not assume it is a server problem.
- The error message tells you which layer failed.
- Use the right tool for the right question.

PAGE 02

VERIQTA

Discover What Network Your Linux Machine Actually Has

Investigate network interfaces, private IP addresses, loopback, subnet prefixes, and default gateways. Explain why a server can have multiple IP addresses.

1. CHECK YOUR NETWORK INTERFACES

```
ip -br addr
```

- Shows all network interfaces with their IP addresses in a short, readable format.
- Helps you quickly see which interfaces are up and what IP addresses they have.

Example output:

```
lo          UNKNOWN    127.0.0.1/8 ::1/128
eth0        UP         192.168.1.10/24
docker0     DOWN       172.17.0.1/16
```

2. SEE DETAILED INTERFACE INFORMATION

```
ip link
```

- Shows detailed information about each network interface.
- Useful for checking the interface state (UP/DOWN), MAC address, and other settings.

Example output (partial):

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP>
   link/ether 02:42:ac:11:00:02 brd ff:ff:ff:ff:ff:ff
   state UP mode DEFAULT qlen 1000
```

3. CHECK YOUR ROUTING TABLE

```
ip route
```

- Shows the routes your machine uses to reach other networks.
- Includes the default gateway for internet access.

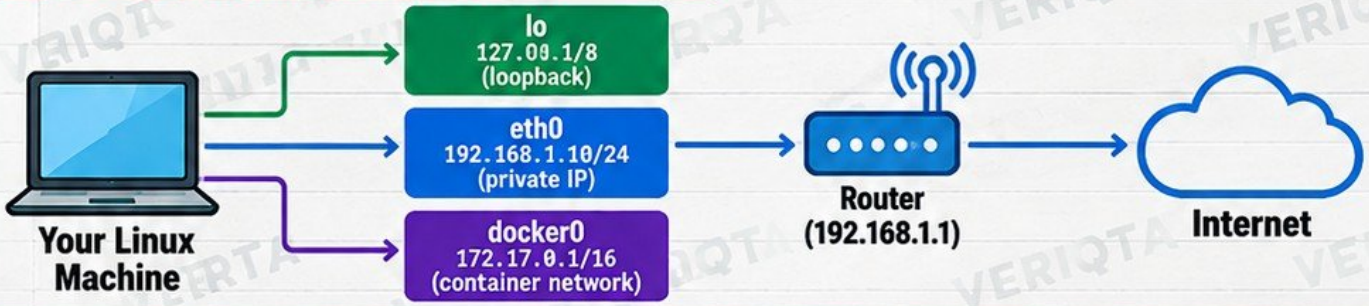
Example output:

```
default via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0 proto kernel
172.17.0.0/16 dev docker0 proto kernel
```

4. UNDERSTAND THE INTERFACES

- **lo (loopback) – 127.0.0.1**
  Used for communication within the same machine.
- **eth0 (or en*) – Private IP**
  Connects to your local network (LAN).
- **docker0 – Docker bridge network**
  Used for container networking.
- **Other interfaces – e.g. tun0, wlan0, ens3**
  Used for VPN, Wi-Fi, or cloud networking.

HOW IT LOOKS IN A REAL NETWORK



WHY A SERVER CAN HAVE MULTIPLE IP ADDRESSES

- Can be on multiple networks (e.g. public and private).
- Can host multiple services that need different IPs.
- Docker and Kubernetes create virtual interfaces.
- Cloud servers often have internal and external IPs.
- Used for security, separation, and network design.

JUNIOR ENGINEER TIPS

- Always identify which interface you are using.
- 127.0.0.1 is only for local communication.
- A private IP (e.g. 192.168.x.x or 10.x.x.x) is not directly reachable from the internet.
- The default route shows how your machine reaches external networks.
- It is normal for a server to have multiple IP addresses.

VERIQTA

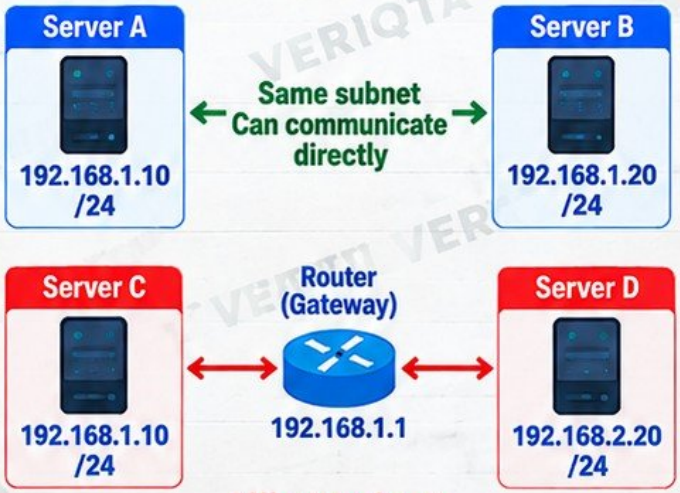
Subnetting Without Guesswork: Can These Servers Communicate?

Use CIDR notation to determine whether two IP addresses belong to the same subnet. Explain when a gateway is needed and why incorrect subnet settings break connectivity.

1. WHAT IS A SUBNET?

- A subnet is a smaller network inside a larger network.
- CIDR notation (e.g. 192.168.1.0/24) shows the network address and the number of bits used for the network portion.
- Devices in the same subnet can communicate directly without a gateway.

2. REAL-WORLD EXAMPLE



3. UNDERSTAND CIDR NOTATION



- /24 means the first 24 bits are the network part.
- This gives 256 addresses (0 – 255).
- Network address: 192.168.1.0
- Usable host range: 192.168.1.1 – 192.168.1.254
- Broadcast address: 192.168.1.255

4. CHECK IF TWO IP ADDRESSES ARE IN THE SAME SUBNET

```
# Using Python ipaddress module
python3 -c "import ipaddress;
net = ipaddress.ip_network('192.168.1.0/24');
print('192.168.1.10 in subnet:';
ipaddress.ip_address('192.168.1.10') in net);
print('192.168.1.20 in subnet:';
ipaddress.ip_address('192.168.1.20') in net)"
```

Output:

```
192.168.1.10 in subnet: True
192.168.1.20 in subnet: True
```

5. CHECK THE ROUTE FOR AN IP

```
# Check how your machine will reach an IP
ip route get 192.168.2.20
```

- Shows whether the destination is on the local network or will be sent to a gateway.
- If it shows "dev eth0" only, it is a local subnet.
- If it shows a gateway (via 192.168.1.1), it is a different subnet.

6. SUBNET EXAMPLES

CIDR	Subnet Mask	Total IPs	Usable IPs
/24	255.255.255.0	256	254
/25	255.255.255.128	128	126
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30

7. WHEN A GATEWAY IS NEEDED

- When the destination IP is in a different subnet.
- When accessing the internet.
- When communicating between private networks.
- The default gateway routes traffic to other networks.

8. COMMON MISTAKES

- ✗ Using the wrong subnet mask.
- ✗ Setting an incorrect gateway.
- ✗ Placing servers in different subnets by mistake.
- ✗ Forgetting to configure a route.
- ✗ Assuming all 192.168.x.x addresses can communicate directly.

9. TRY THIS (LAB)

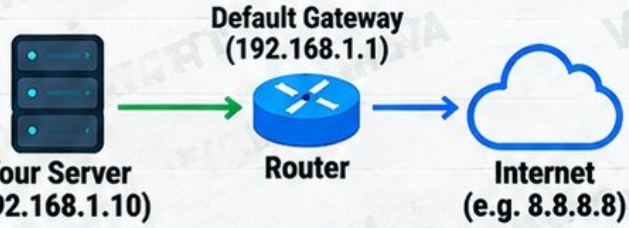
1. Use Python `ipaddress` to check if `10.0.0.10` and `10.0.0.20` are in the same subnet (/24).
2. Run `ip route get 8.8.8.8` and observe the gateway used.
3. Try different CIDR values (e.g. /24, /25, /26) and calculate the network, usable range, and broadcast address.

VERIQTA

A Server Has an IP Address but Cannot Reach the Internet

Investigate missing default routes, unreachable gateways, and routing decisions. Distinguish a routing problem from a DNS failure.

1. UNDERSTAND THE PATH TO THE INTERNET



- Your server sends traffic to a default gateway.
- The gateway routes it to the internet.
- If the default route is missing or the gateway is unreachable, the server cannot reach external networks.

2. CHECK YOUR IP ADDRESS

```
ip -br addr
```

Example output:

```
lo       UNKNOWN   127.0.0.1/8   ::1/128
eth0     UP        192.168.1.10/24
```

- Confirms the server has an IP address.
- Shows the network interface (e.g. eth0).
- The /24 means the subnet mask is 255.255.255.0.

3. CHECK THE ROUTING TABLE

```
ip route
```

Example output (good):

```
default via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0 proto kernel
169.254.0.0/16 dev eth0 scope link
```

Example output (missing default route):

```
192.168.1.0/24 dev eth0 proto kernel
169.254.0.0/16 dev eth0 scope link
```

- The default route (0.0.0.0/0 or default) tells the server where to send internet traffic.
- If it is missing, the server can only reach its local network.

4. CHECK THE ROUTE TO A SPECIFIC IP

```
ip route get 8.8.8.8
```

Example output (working):

```
8.8.8.8 via 192.168.1.1 dev eth0
src 192.168.1.10 uid 0
```

Example output (no route):

```
RTNETLINK answers: Network is unreachable
```

- Shows which gateway and interface will be used for the destination IP.
- If you get "Network is unreachable", there is no route to that IP.

5. TEST CONNECTIVITY STEP BY STEP

`ping 192.168.1.1`	Test if the gateway is reachable.
`ping 8.8.8.8`	Test if you can reach an external IP.
`ping example.com`	Test DNS and full connectivity.
`tracepath 8.8.8.8`	Show the network path to the destination.
`tracepath example.com`	Identify where the connection stops.

6. INTERPRET THE RESULTS

Gateway reachable, external IP fails	Possible firewall or ISP issue.
Gateway unreachable	Wrong gateway, cable issue or local network problem.
External IP works, domain name fails	DNS problem (name resolution).
Both fail	Missing default route or network blocking traffic.

7. COMMON CAUSES

- ✗ Missing default route.
- ✗ Incorrect gateway IP.
- ✗ Gateway is down or unreachable.
- ✗ Firewall or security group blocking outbound traffic.
- ✗ Wrong subnet configuration.
- ✗ DNS server misconfiguration (affects domain names only).

8. FIX EXAMPLES

`sudo ip route add default via 192.168.1.1`	Add a default route (temporary).
`sudo ip route replace default via 192.168.1.1`	Replace existing route.
`sudo ip route del default`	Remove default route.

9. TRY THIS (LAB)

1. Run `ip route` and check if you have a default route.
2. Ping your gateway, then `8.8.8.8`, then `example.com`.
3. Use `tracepath 8.8.8.8` to see where the connection stops.
4. Remove the default route (in a safe lab environment) and observe the failure.

VERIQTA

Your Application Is Running, but Nobody Can Connect

Start a local HTTP server. Investigate listening ports, bind addresses, and why 127.0.0.1 differs from 0.0.0.0. Learn when localhost-only binding is intentional and secure.

1. START A LOCAL HTTP SERVER

```
python3 -m http.server 8000
```

- Starts a simple web server on port 8000.
- By default, it binds to 0.0.0.0 (all interfaces).
- You can also bind to a specific address.

Bind to a specific address:

```
python3 -m http.server 8000 --bind 127.0.0.1
python3 -m http.server 8000 --bind 0.0.0.0
```

2. CHECK LISTENING PORTS

```
ss -lntp | grep 8000
```

- Shows which address and port the application is listening on.
- Confirms if it is bound to 127.0.0.1 or 0.0.0.0.

Example output:

```
State    Local Address:Port    Peer Address    Process
LISTEN   127.0.0.1:8000        0.0.0.0:*       users:(("python3",pid=1234))
LISTEN   0.0.0.0:8000          0.0.0.0:*       users:(("python3",pid=1234))
```

3. UNDERSTAND BIND ADDRESSES

127.0.0.1 (localhost)

- Accepts connections only from the same machine.
- Not reachable from other devices on the network.
- Often used intentionally for security (e.g. database, admin UI).

0.0.0.0 (all interfaces)

- Listens on all network interfaces.
- Reachable from other devices (if firewall allows).
- Used when you want the application to be accessible on the network.

4. TEST CONNECTIVITY

Test from the same machine

```
curl http://127.0.0.1:8000
```

- Should work even if the server is bound to 127.0.0.1.

Test using the server's IP address

```
curl http://<server-ip>:8000
```

- Works only if the server is bound to 0.0.0.0 and firewall allows the connection.
- Fails if the server is bound to 127.0.0.1.

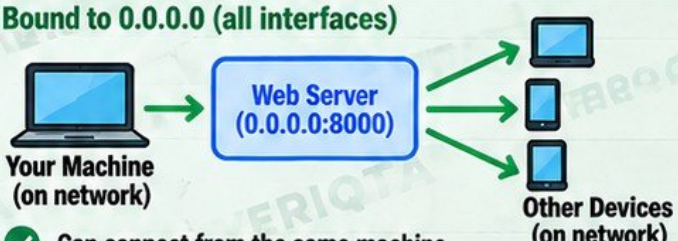
5. VISUAL EXAMPLE

Bound to 127.0.0.1 (localhost only)



- ✅ Can connect: `curl http://127.0.0.1:8000`
- ❌ Cannot connect: `curl http://<server-ip>:8000`

Bound to 0.0.0.0 (all interfaces)



- ✅ Can connect from the same machine
- ✅ Can connect from other devices (if firewall allows)

6. COMMON PROBLEMS

- ❌ Application is bound to 127.0.0.1, so remote devices cannot connect.
- ❌ Firewall or security group is blocking the port.
- ❌ Using the wrong IP address or port.
- ❌ Application is not running or has crashed.

7. TROUBLESHOOTING STEPS

1. Check if the application is running.
2. Verify the listening address and port (`ss -lntp`).
3. Test with `curl` from local and remote machines.
4. Check firewall rules (`ufw`, `nft`, or cloud security group).

JUNIOR ENGINEER TIP

- Use 127.0.0.1 for services that should not be publicly accessible (e.g. database).
- Use 0.0.0.0 only when you intentionally want other devices to connect, and ensure proper firewall rules.

VERIQTA

Connection Refused, Timeout, or Unreachable: Read the Evidence

Reproduce a refused connection and compare it with a timeout and routing failure. Understand what each symptom suggests without assuming a single cause.

1. CONNECTION REFUSED



Client (your machine) — Server (host is reachable but port is closed)

- The host is reachable, but no application is listening on the port.
- You will get an immediate response saying the connection is refused.
- Common when the service is not running or listening on a different port.

2. CONNECTION TIMEOUT



Client (your machine) — Server (no response)

- No response is received from the destination.
- Packets are sent but there is no reply.
- Could be caused by a firewall blocking the traffic, or the host is down, or the network is filtering the packets.

3. NETWORK UNREACHABLE



Client (your machine) — Router (cannot reach network) — Destination (network unreachable)

- Your machine cannot reach the destination network.
- Usually caused by missing routes, incorrect gateway, or network configuration issues.
- You will get an error like "Network is unreachable" or "No route to host".

4. TEST WITH CURL

```
curl --connect-timeout 5 http://192.168.1.10:8080
```

Example output (connection refused):

```
curl: (7) Failed to connect to 192.168.1.10 port 8080: Connection refused
```

5. TEST WITH NETCAT (NC)

```
nc -vz 192.168.1.10 0880
```

Example output (timeout):

```
nc: connect to 192.168.1.10 port 8080 (tcp) timed out: Operation timed out
```

6. CHECK LISTENING PORTS

```
ss -lntp | grep 8080
```

Example output (nothing listening):

- No output means no application is listening on port 8080.
- If the application is running but not listening, it will not accept connections.

7. COMPARE COMMON ERROR MESSAGES

Symptom / Error Message	What It Means	Likely Causes
Connection refused (curl: (7) Connection refused)	The host is reachable, but no application is listening on the port.	• Application not running. • Application listening on a different port. • Binding only to 127.0.0.1 (localhost).
Operation timed out (curl: (28) Operation timed out)	No response received from the destination.	• Firewall blocking the traffic. • Host is down or not reachable. • Network filtering or security group rules.
Network is unreachable (curl: (6) Could not resolve host / No route to host / Network unreachable)	Your machine cannot reach the destination network.	• Missing default route or incorrect gateway. • Wrong network configuration. • Destination network does not exist or is blocked.

8. TROUBLESHOOTING STEPS

1. Check if the destination host IP is reachable (ping).
2. Check if the port is open (nc -vz or curl).
3. Verify if the application is running and listening (ss -lntp).
4. Check local routing table (ip route).
5. If it is a remote host, check firewall or security group rules.
6. Compare the error message with the table above.

9. TRY THIS (LAB)

1. Test a port that is closed to see "connection refused".
2. Test a non-routable IP (e.g. 10.255.255.1) to see a timeout.
3. Test an invalid network (e.g. 203.0.113.1) to see "network unreachable".
4. Use curl --connect-timeout, nc -vz, and ss to compare the results.
5. Document the outputs and explain the cause for each case.

JUNIOR ENGINEER TIP

- Read the exact error message before making changes.
- Do not assume it is a DNS problem.
- A connection refused error means the host is reachable.
- A timeout usually means no response.
- A network unreachable error means a routing issue.
- Always test each layer separately: DNS → Network → Port → Application.

VERIQTA

DNS Says One Thing, but Your Application Reaches Another

Examine DNS records, local hostname overrides, resolver behavior, caching, and how applications can receive different DNS answers.

1. HOW DNS RESOLUTION WORKS



- Your machine asks a DNS resolver for an IP address.
- The resolver may use cache or query other DNS servers.
- It returns an IP address to your machine.
- Applications then use that IP address to make a connection.

2. CHECK DNS RECORDS WITH DIG

```
dig example.com
```

- Shows the DNS records returned by the configured resolver.
- Useful for checking A, AAAA, CNAME, TTL and the actual IP addresses.
- Helps you verify if DNS is returning the expected answer.

Example output (shortened):

```
$ dig example.com
;; ANSWER SECTION:
example.com.    300  IN  A      93.184.216.34
example.com.    300  IN  AAAA   2606:2800:220:1:248:1893:25c8:1946
```

3. CHECK RESOLUTION USING GETENT

```
getent hosts example.com
```

- Uses the system's configured name resolution method (e.g. /etc/hosts, DNS, etc).
- Shows the IP address that your systemt will actually use.
- Useful to compare with dig to see if there is a difference.

Example output:

```
93.184.216.34   example.com
2606:2800:220:1:248:1893:25c8:1946   example.com
```

4. VIEW YOUR DNS RESOLVER CONFIGURATION

```
resolvectl status
```

- Shows your current DNS servers and resolver settings.
- Useful to verify which DNS server your system is using (e.g. ISP, company, or custom).

Example output (partial):

```
Global
   DNS Servers: 1.1.1.1 8.8.8.8
   DNS Domain:  ~.
Link 2 (eth0)
   Current Scopes: DNS
   DNS Servers: 192.168.1.1
```

5. LOCAL HOSTNAME OVERRIDES (/etc/hosts)

```
cat /etc/hosts
```

- The /etc/hosts file can override DNS records.
- If a hostname exists here, your system will use this IP instead of the DNS response.
- This can cause your application to reach a different server.

Example output:

```
127.0.0.1       localhost
192.168.1.100   example.com      # overides DNS
::1             localhost
```

6. TEST APPLICATION BEHAVIOR WITH CURL

```
curl --resolve example.com:80:192.168.1.100 http://example.com
```

- Forces curl to use a specific IP address for a hostname, ignoring normal DNS resolution.
- Useful to test if the application can reach a different server and to compare behavior.

Example output:

```
* Added example.com:80:192.168.1.100 to DNS cache
* Connected to example.com (192.168.1.100) port 80
> GET / HTTP/1.1
> Host: example.com
```

7. WHY YOU MAY SEE DIFFERENT DNS ANSWERS

Reason	Description
Local /etc/hosts entry	Overrides DNS and returns a different IP.
Different DNS server	Your ISP, company, or VPN may return different answers.
DNS caching	Your system or a DNS server may have a cached old record.
Load balancing	DNS may return different IPs (multiple A/AAAA records).
Split DNS	Internal and external networks return different IPs for the same hostname.

8. TROUBLESHOOTING STEPS

1. Check /etc/hosts for local overrides.
2. Use `dig` to see the actual DNS records.
3. Use `getent hosts` to see what your system resolves.
4. Check your resolver configuration with `resolvectl`.
5. Use `curl --resolve` to test a specific IP.
6. Compare the results and identify why they differ.

9. JUNIOR ENGINEER TIPS

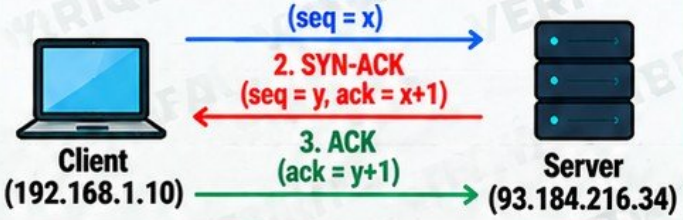
- Always check /etc/hosts first.
- Do not assume DNS is the only source of hostname resolution.
- DNS records can be different on internal and external networks.
- Caching can make old records persist for a while.
- Use multiple tools (`dig`, `getent`, curl) to compare results.
- Document what you find before making changes.

VERIQTA

Watch TCP Build a Connection and See What UDP Does Differently

Observe TCP's three-way handshake and connection states. Compare UDP's connectionless behavior and why it is used for services such as traditional DNS queries.

1. TCP THREE-WAY HANDSHAKE



- Client sends SYN to start a connection.
- Server responds with SYN-ACK.
- Client sends ACK and the connection is established.
- After the handshake, data is sent in both directions.

2. TCP CONNECTION STATES (ss -tan)

```
$ ss -tan | grep :80
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port
SYN-SENT	0	1	192.168.1.10:52344	93.184.216.34:80
SYN-RECV	0	1	192.168.1.10:52344	93.184.216.34:80
ESTAB	0	0	192.168.1.10:52344	93.184.216.34:80
TIME-WAIT	0	0	192.168.1.10:52344	93.184.216.34:80
CLOSE-WAIT	0	0	192.168.1.10:52344	93.184.216.34:80

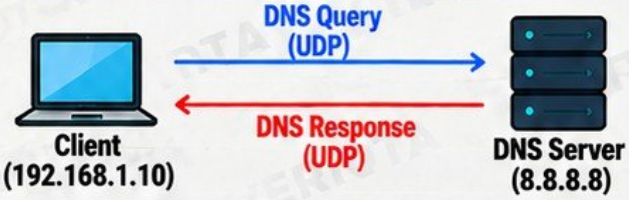
- SYN-SENT: client sent SYN, waiting for response.
- SYN-RECV: server received SYN, sent SYN-ACK.
- ESTAB: connection established.
- TIME-WAIT: connection closed, waiting to ensure all packets are received.
- CLOSE-WAIT: remote side closed, waiting for local process to close.

3. CAPTURE THE TCP HANDSHAKE WITH TCPDUMP

```
$ sudo tcpdump -ni any tcp port 80 and host 93.184.216.34
12:10:45.123456 IP 192.168.1.10.52344 > 93.184.216.34.80:
    Flags [S], seq 123456789, win 64240, length 0
12:10:45.234567 IP 93.184.216.34.80 > 192.168.1.10.52344:
   Flags [S.], seq 987654321, ack 123456790, win 65160, length 0
12:10:45.234789 IP 192.168.1.10.52344 > 93.184.216.34.80:
    Flags [.], ack 987654322, win 64240, length 0
```

- You can see the SYN, SYN-ACK and ACK packets.
- After the handshake, you will see packets with data (HTTP request/response).

4. UDP IS CONNECTIONLESS



- UDP does not have a handshake.
- The client sends a request and the server responds.
- There is no connection state, no SYN, ACK, or retransmission.
- This makes UDP fast and lightweight, which is why it is used for traditional DNS queries.

5. TEST DNS OVER UDP (DEFAULT)

```
$ dig example.com +notcp
; <<>> DiG 9.18.1 <<>> example.com +notcp
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 12345
;; flags: qr rd ra; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 1
```

- Uses UDP (default) to query DNS.
- Faster and suitable for most queries.

6. TEST DNS OVER TCP

```
$ dig example.com +tcp
; <<>> DiG 9.18.1 <<>> example.com +tcp
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 54321
;; flags: qr rd ra; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 1
```

- Uses TCP to query DNS.
- Slower due to TCP handshake, but required for large responses (such as DNS zone transfers) or when UDP is blocked.

7. COMPARE TCP AND UDP

Feature	TCP	UDP
Connection	Connection-oriented (uses handshake)	Connectionless (no handshake)
Reliability	Reliable (retransmits lost packets)	No built-in reliability
Speed	Slower (handshake overhead)	Faster (no handshake)
Use cases	Web (HTTP/HTTPS), SSH, databases, file transfer	DNS (traditional), streaming, VoIP, real-time services

8. WHAT TO LOOK FOR IN PACKET CAPTURES

1. SYN, SYN-ACK, ACK for TCP handshake.
2. Source and destination IP addresses and ports.
3. Flags (SYN, ACK, FIN, RST).
4. Sequence and acknowledgment numbers.
5. Request and response data (HTTP, DNS, etc).
6. For UDP, only request and response packets (no handshake).

9. JUNIOR ENGINEER TIPS

- Use `tcpdump` with filters to avoid unnecessary noise.
- Use `ss -tan` to check connection states.
- Use `dig +notcp` and `dig +tcp` to compare DNS over UDP and TCP.
- If DNS over UDP fails but TCP works, UDP might be blocked or the response is too large.
- Always check both the client and server side when troubleshooting network issues.

VERIQTA

HTTP Works, HTTPS Fails: Find the Broken Layer

Investigate TLS certificate validation, hostname mismatches, trust chains, and failed handshakes. Understand why bypassing certificate verification is not a production fix.

1. VERIFY HTTP WORKS

```
curl -vI http://example.com
```

```
> GET / HTTP/1.1
> Host: example.com
>
< HTTP/1.1 301 Moved Permanently
< Location: https://example.com/
```

- Confirms the server is reachable over HTTP.
- Helps you determine if the problem is specific to HTTPS (TLS) and not general connectivity.
- Many websites redirect HTTP to HTTPS.

2. CHECK HTTPS AND READ THE ERROR

```
curl -vI https://example.com
```

```
* TLS handshake failed
* SSL certificate problem:
  unable to get local issuer certificate
curl: (60) SSL certificate problem:
  unable to get local issuer certificate
```

- Shows the exact TLS error message.
- Common errors include:
  - certificate has expired
  - unable to get local issuer certificate
  - hostname mismatch
  - self-signed certificate in certificate chain

3. INSPECT THE TLS CERTIFICATE WITH OPENSSL

```
openssl s_client -connect example.com:443 -servername example.com
```

```
Certificate chain
 0 s:CN = example.com
   i:C = US, O = Example CA, CN = Example Root CA
---
Verify return code: 0 (ok)
```

- Shows the full certificate chain.
- Helps you check the issuer, validity dates, and hostname.
- Verify return code: 0 (ok) means the certificate is trusted.

4. CHECK CERTIFICATE DETAILS

```
openssl x509 -in cert.pem -noout -text
```

```
Certificate:
   Subject: CN = example.com
   Issuer: C = US, O = Example CA, CN = Example Root CA
   Validity
      Not Before: Oct 10 00:00:00 2024 GMT
      Not After : Oct 10 23:59:59 2025 GMT
   Subject Alternative Name:
      DNS:example.com, DNS:www.example.com
```

- Check the Common Name (CN) and Subject Alternative Name (SAN).
- Verify the certificate is not expired.
- Ensure the hostname you are connecting to is listed in the SAN.

5. COMMON TLS/HTTPS FAILURES

Problem	Example Error Message	What It Means
Expired certificate	`certificate has expired`	The certificate is no longer valid.
Hostname mismatch	`certificate is not valid for 'example.com'`	The domain name does not match the certificate.
Untrusted issuer	`unable to get local issuer certificate`	The certificate chain is not trusted by your system.
Self-signed cert	`self-signed certificate in certificate chain`	The server is using a self-signed certificate.
Incomplete chain	`unable to verify the first certificate`	The server did not send the full certificate chain.
Protocol or cipher mismatch	`handshake failure`	The client and server cannot agree on TLS version or cipher.

6. COMPARE A SUCCESSFUL TLS HANDSHAKE

```
openssl s_client -connect example.com:443 -servername example.com
```

```
CONNECTED(00000003)
...
SSL-Session:
    Protocol  : TLSv1.3
    Cipher    : TLS_AES_256_GCM_SHA384
    Verify return code: 0 (ok)
```

- Shows the negotiated TLS version and cipher.
- Verify return code: 0 (ok) means the certificate is valid and trusted.
- If the handshake fails, the problem is in the TLS layer (e.g. certificate, protocol, or trust chain).

7. WHY YOU SHOULD NOT BYPASS CERTIFICATE VERIFICATION

```
curl -k https://example.com
```

- ✗ The `-k` / `--insecure` option ignores certificate errors.
- ✗ It exposes you to man-in-the-middle (MITM) attacks.
- ✗ It does not fix the real problem.
- ✗ It should NEVER be used in production.
- ✗ Always fix the certificate issue properly (e.g. correct certificate, proper hostname, trusted CA, complete chain).

8. TROUBLESHOOTING STEPS

1. Confirm HTTP works and the issue is specific to HTTPS.
2. Check the exact error message from `curl -vI`.
3. Inspect the certificate with `openssl s_client`.
4. Verify the hostname and SAN match.
5. Check if the certificate is expired.
6. Ensure the certificate chain is complete and trusted.
7. Check for protocol or cipher mismatches.
8. Fix the root cause instead of bypassing verification.

9. JUNIOR ENGINEER TIPS

- ✓ Always read the full error message.
- ✓ Check both the certificate and the chain.
- ✓ Verify the hostname matches the certificate.
- ✓ Understand the difference between certificate errors and network connectivity issues.
- ✓ Never use `--insecure` in production.
- ✓ Document the issue and the fix for future reference.

VERIQTA

Build a Reverse Proxy and Diagnose a 502 Bad Gateway

Put Nginx in front of a local application. Break the upstream connection deliberately and inspect the evidence. Distinguish proxy failures from application errors.

1. ARCHITECTURE OVERVIEW



- Clients send requests to Nginx (port 80).
- Nginx forwards the requests to the local application (port 8000).
- Nginx returns the application's response to the client.
- If Nginx cannot reach the upstream application, it returns 502 Bad Gateway.

2. CONFIGURE NGINX AS A REVERSE PROXY

`/etc/nginx/conf.d/app.conf`

```
server {
    listen 80;
    server_name localhost;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For;
        $proxy_add_x_forwarded_for;
    }
}
```

- This configuration sends all requests to a local application running on port 8000.
- Save the file and reload Nginx:

```
sudo nginx -t && sudo systemctl reload nginx
```

3. START A LOCAL APPLICATION

Use a simple Python web server:

```
python3 -m http.server 8000
```

- The application will be available on http://127.0.0.1:8000
- It returns a simple web page.

TEST THROUGH NGINX

```
curl -v http://localhost/
```

- You should see a successful response from the application through Nginx.
- Nginx is acting as a reverse proxy.

4. BREAK THE UPSTREAM CONNECTION

Stop the application deliberately:

```
sudo pkill -f "python3 -m http.server"
```

- Now the application on port 8000 is not running.

5. OBSERVE THE 502 BAD GATEWAY

Test again:

```
curl -v http://localhost/
```

```
> GET / HTTP/1.1
> Host: localhost
>
< HTTP/1.1 502 Bad Gateway
< Server: nginx/1.24.0
< Content-Type: text/html
...
<html>
<head><title>502 Bad Gateway</title></head>
```

- Nginx returns 502 because it cannot connect to the upstream application.
- This is a proxy failure, not an application error.

6. CHECK NGINX ERROR LOGS

View the error log:

```
sudo tail -n 20 /var/log/nginx/error.log
```

```
2026/10/28 10:15:23 [error] 12345#12345:
connect() failed (111: Connection refused)
while connecting to upstream,
client: 127.0.0.1, server: localhost,
request: "GET / HTTP/1.1",
upstream: "http://127.0.0.1:8000/",
host: "localhost"
```

- The log shows Nginx could not connect to 127.0.0.1:8000 (connection refused).
- This confirms the issue is with the upstream application, not the client.

7. USE ss TO VERIFY THE PORT STATUS

Check if the application is listening:

```
ss -lntp | grep 8000
```

When the application is running:

```
State    Recv-Q Send-Q  Local Address:Port     Process
LISTEN   0      128     127.0.0.1:8000    users:("python3",pid=1234)
```

When the application is stopped:

```
ss -lntp | grep 8000
```

- No output means nothing is listening on port 8000.

8. DISTINGUISH PROXY FAILURE FROM APPLICATION ERROR

Scenario	Status Code	Meaning
Nginx cannot reach upstream (application down)	502 Bad Gateway	Proxy failure (cannot connect to upstream).
Application returns an error (e.g. crash or exception)	500 Internal Server Error	Application error (upstream responded with an error).
Application not found	404 Not Found	Application responded but resource does not exist.
Nginx misconfigured	502 / 504	Proxy configuration or network issue.

9. TROUBLESHOOTING STEPS

1. Check if the upstream application is running.
2. Verify the application is listening on the correct port (ss -lntp).
3. Check Nginx error logs for connection errors.
4. Confirm the proxy_pass URL and port are correct.
5. Test the application directly (curl http://127.0.0.1:8000).
6. Check firewall or security group rules (if remote).

10. JUNIOR ENGINEER TIPS

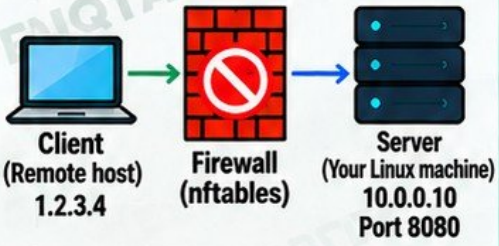
- A 502 error means Nginx could not get a valid response from the upstream server.
- Always check if the application is running and listening on the expected port.
- Use Nginx error logs to find the exact reason for the failure.
- Do not assume 502 is an application bug — it is usually a proxy to upstream connectivity issue.
- Compare with 500 errors, which come from the application itself.

VERIQTA

A Firewall Blocks Traffic: Prove Where the Block Happens

Explore ingress and egress filtering, stateful connections, and why an open listening port does not guarantee remote reachability. Test a temporary rule safely in an isolated lab.

1. TRAFFIC FLOW OVERVIEW



- Traffic must pass through firewall rules before it reaches your server.
- Both ingress (incoming) and egress (outgoing) rules can block traffic.
- A listening port (ss) only shows the application is ready, not that the firewall allows remote access.

2. CHECK LISTENING PORTS

```
ss -lntp | grep 8080
```

```
LISTEN     0      128
0.0.0.0:8080      0.0.0.0:*
users:("python3",pid=1234,fd=3)

LISTEN     0      128
[::]:8080         [::]:*
users:("python3",pid=1234,fd=3)
```

- Confirms your application is listening on the port.
- Does not mean the port is reachable from another machine.
- If it shows only 127.0.0.1, it is bound to localhost only.

3. INSPECT FIREWALL RULES

```
sudo nft list ruleset
```

```
table inet filter {
  chain input {
    type filter hook input priority 0;
    policy drop;
    ct state established,related accept
    iif "lo" accept
    tcp dport 22 accept
    tcp dport 8080 accept
  }
  chain output {
    type filter hook output priority 0;
    policy accept;
  }
}
```

- Shows current firewall rules (nftables).
- Look for default policies (accept or drop).
- Check which ports and sources are allowed.

4. TEST CONNECTIVITY FROM A REMOTE HOST

```
curl -v http://<server-ip>:8080
```

```
*   Trying 10.0.0.10:8080...
* connect to 10.0.0.10 port 8080 failed:
  Connection timed out
curl: (28) Connection timed out
```

- Connection timed out usually means the firewall is dropping the packets.
- Connection refused means the host is reachable but the application is not listening.
- If it works locally but not remotely, the block is likely at the firewall or network level.

5. CHECK FIREWALL COUNTERS

```
sudo nft list ruleset -a
```

```
chain input {
  type filter hook input priority 0;
  policy drop;
  ct state established,related accept
  iif "lo" accept
  tcp dport 8080 counter packets 0 bytes 0
  tcp dport 22 counter packets 124 bytes 12K
}
```

- Counters show how many packets match each rule.
- If the counter for port 8080 does not increase when you test, the packet is being dropped before this rule.

6. TEST A TEMPORARY RULE SAFELY

```
sudo nft add rule inet filter input \
  tcp dport 8080 ip saddr 1.2.3.4 accept
```

- This temporarily allows traffic from a specific IP address.
- Test the connection again from that IP.
- Remove the rule after testing:

```
sudo nft delete rule inet filter input \
  tcp dport 8080 ip saddr 1.2.3.4 accept
```

⚠ Only test in an isolated lab or controlled environment. Do not open your server to the internet unnecessarily.

7. COMMON SYMPTOMS AND WHAT THEY MEAN

Symptom	What It Means	Likely Cause
Connection timed out	No response from server	Firewall dropping packets, network ACL, or routing issue
Connection refused	Host reachable but port closed	Application not running or firewall sending reject
Works locally but not remotely	Service is running on the server	Firewall blocking external traffic or wrong bind address
No counter increase on firewall rule	Packet not reaching that rule	Blocked earlier in the network path (cloud firewall, router, etc.)

8. TROUBLESHOOTING STEPS

1. Verify the application is listening (ss -lntp).
2. Test from a remote host (curl).
3. Check firewall rules (nft list ruleset).
4. Inspect firewall counters (-a).
5. Identify whether the block is ingress or egress.
6. Add a temporary rule to test in an isolated lab.
7. Remove the rule after testing.
8. Check other network controls (cloud security group, router ACL).

9. JUNIOR ENGINEER TIPS

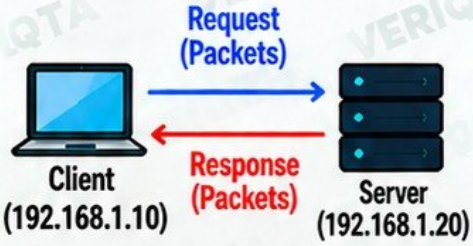
- A listening port does not guarantee remote reachability.
- Always check firewall rules and counters.
- Test from both local and remote machines.
- Understand the difference between timeout and connection refused.
- Use temporary, specific rules for testing.
- Do not leave test rules open.
- Remember there may be multiple firewalls (server, cloud, network).

VERIQTA

Capture Packets Without Drowning in Network Noise

Use packet captures to determine whether requests reach the server and whether replies leave it. Understand capture interfaces, filters, and what encrypted traffic hides.

1. HOW PACKET CAPTURE WORKS



- A packet capture records network traffic that passes through a network interface.
- It helps you see if requests reach the server and if replies leave it.
- You can filter the capture to focus only on the traffic you care about.
- Encrypted traffic (e.g. HTTPS) hides the content but you can still see IPs, ports, and handshake information.

2. CAPTURE TRAFFIC WITH TCPDUMP

```
sudo tcpdump -ni eth0
```

```
tcpdump: verbose output suppressed,
use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB
(Ethernet), snapshot length 262144 bytes
12:10:15.123456 IP 192.168.1.10.52344 >
192.168.1.20.80: Flags [S], seq 123456789,
win 64240, length 0
12:10:15.123789 IP 192.168.1.20.80 >
192.168.1.10.52344: Flags [S.],
seq 987654321, ack 123456790, win 65160,
length 0
```

- `-ni` shows live capture with interface and disables name resolution (clean output).
- Shows the actual packets going in and out.
- Useful to confirm if the server receives the request and sends a response.

3. USE CAPTURE FILTERS

Filter	Description	Example
host	Capture traffic to or from a specific IP	`tcpdump -ni eth0 host 192.168.1.20`
port	Capture traffic on a specific port	`tcpdump -ni eth0 port 80`
tcp	Capture only TCP traffic	`tcpdump -ni eth0 tcp`
udp	Capture only UDP traffic	`tcpdump -ni eth0 udp`
and	Combine filters	`tcpdump -ni eth0 tcp port 80 and host 192.168.1.20`
not	Exclude traffic	`tcpdump -ni eth0 not port 22`

- Filters reduce noise and show only the traffic you are interested in.
- You can combine multiple filters.
- This makes it easier to analyze the capture.

4. CAPTURE AND ANALYZE IN WIRESHARK



1. Open Wireshark and select the correct network interface (e.g. eth0).
2. Start the capture and reproduce the issue (e.g. make a request).
3. Use display filters (e.g. ip.addr == 192.168.1.20) to focus on the traffic.
4. Inspect the packets, timing, and flags.
5. Compare requests and responses to confirm where the problem is.

5. EXAMPLE: CAPTURE HTTP TRAFFIC

```
sudo tcpdump -ni eth0 tcp port 80
```

```
12:11:01.111111 IP 192.168.1.10.53000 >
192.168.1.20.80: Flags [S], seq 1000,
win 64240, length 0
12:11:01.111222 IP 192.168.1.20.80 >
192.168.1.10.53000: Flags [S.], seq 2000,
ack 1001, win 65160, length 0
12:11:01.111333 IP 192.168.1.10.53000 >
192.168.1.20.80: Flags [P.], seq 1001,
ack 2001, win 64240, length 349
```

- Shows the TCP handshake and HTTP request.
- Helps you verify that the request reaches the server and a response is sent back.

6. EXAMPLE: CAPTURE HTTPS TRAFFIC

```
sudo tcpdump -ni eth0 tcp port 443
```

```
12:12:05.555555 IP 192.168.1.10.54000 >
192.168.1.20.443: Flags [S], seq 3000,
win 64240, length 0
12:12:05.555777 IP 192.168.1.20.443 >
192.168.1.10.54000: Flags [S.], seq 4000,
ack 3001, win 65160, length 0
12:12:05.556000 IP 192.168.1.10.54000 >
192.168.1.20.443: Flags [P.], seq 3001,
ack 4001, win 64240, length 517
[Encrypted TLS data]
```

- You can see the TLS handshake and packet sizes.
- The actual content is encrypted and cannot be read without decryption keys.

7. KEY THINGS TO LOOK FOR

What to Check	What It Tells You
Do you see the request packet?	If not, the traffic is being blocked before it reaches the server.
Do you see the response packet?	If not, the server may not be replying or the reply is being blocked on the way back.
TCP flags (S, S., P, R)	Helps you understand the connection state and errors.
Packet timing	Delays or retransmissions can indicate network issues.
ICMP messages	May show unreachable hosts, TTL expired, or other errors.

8. USEFUL CAPTURE FILTER EXAMPLES

Scenario	Filter Command
HTTP traffic to a server	`tcpdump -ni eth0 tcp port 80 and host 192.168.1.20`
HTTPS traffic to a server	`tcpdump -ni eth0 tcp port 443 and host 192.168.1.20`
Only traffic from a client IP	`tcpdump -ni eth0 host 192.168.1.10`
Exclude SSH traffic	`tcpdump -ni eth0 not port 22`
Capture DNS (UDP port 53)	`tcpdump -ni eth0 udp port 53`
Capture and save to a file	`tcpdump -ni eth0 -w capture.pcap`

9. TROUBLESHOOTING STEPS

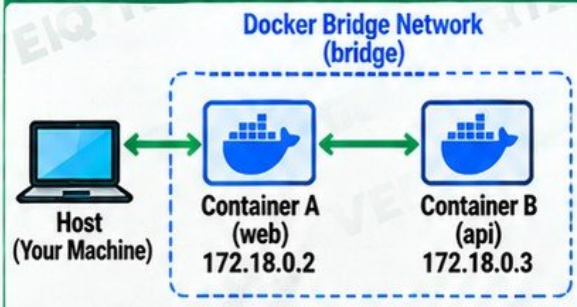
1. Identify the correct network interface.
2. Start a capture with an appropriate filter.
3. Reproduce the issue (make the request).
4. Check if the request reaches the server.
5. Check if the response leaves the server.
6. Look for error packets (ICMP, TCP RST).
7. Analyze timing and retransmissions.
8. Narrow the filter if there is too much noise.

VERIQTA

Why Localhost Changes Meaning Inside a Docker Container

Run two containers and investigate network namespaces, bridge networking, container IPs, and service-name resolution. Show why localhost inside a container is not the host or another container.

1. CONTAINER NETWORKING OVERVIEW



- Containers on the same bridge network can reach each other using their container names.
- Each container has its own network namespace and its own localhost (127.0.0.1).
- localhost inside a container refers to the container itself, not the host and not another container.

2. CREATE TWO CONTAINERS

```
# Create a user-defined network
docker network create demo-net

# Run a simple web server in container A
docker run -d --name web \
  --network demo-net \
  -p 8080:80 \
  nginx:alpine

# Run a second container with a shell
docker run -it --name api \
  --network demo-net \
  alpine:latest sh
```

- Both containers are connected to the same network (demo-net).
- Container A runs Nginx on port 80.
- Container B gives us a shell to test connectivity.

3. INSPECT THE NETWORK

```
docker network inspect demo-net
[
  {
    "Name": "demo-net",
    "Driver": "bridge",
    "Containers": {
      "web": {
        "IPv4Address": "172.18.0.2/16"
      },
      "api": {
        "IPv4Address": "172.18.0.3/16"
      }
    }
]
```

- Shows the network, driver and container IP addresses.
- Confirms both containers are on the same bridge network and can reach each other using container names or IPs.

4. TEST CONNECTIVITY FROM CONTAINER B

```
# Access container A using its name
docker exec -it api sh -c "curl -I http://web"

HTTP/1.1 200 OK
Server: nginx/1.25.3
Date: Tue, 28 Oct 2026 12:00:00 GMT
Content-Type: text/html
Content-Length: 615

# Access container A using its IP
docker exec -it api sh -c "curl -I http://172.18.0.2"
```

- Containers can reach each other using container names (web) or their IP addresses.
- Docker provides built-in DNS for container name resolution.

5. WHY LOCALHOST IS DIFFERENT

```
# Inside container B
docker exec -it api sh-c "curl -I http://localhost"

curl: (7) Failed to connect to localhost
    port 80: Connection refused

# Check what localhost is
docker exec -it api sh -c "ip addr show lo"
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536
   inet 127.0.0.1/8 scope host lo
   inet6 ::1/128 scope host
```

- localhost (127.0.0.1) inside the container refers to the container itself.
- It does not point to the host and does not reach other containers.

6. ACCESS THE HOST FROM A CONTAINER

```
# On Docker (Linux)
docker exec -it api sh -c "curl -I \
  http://host.docker.internal:8080"

# Or use the host gateway IP
docker exec -it api sh -c "ip route"

default via 172.18.0.1 dev eth0
```

- On Linux, the host is usually at 172.18.0.1 (the bridge gateway).
- On Docker Desktop (Mac/Windows), use host.docker.internal.
- This allows a container to reach services running on the host machine.

7. KEY CONCEPTS TO REMEMBER

Concept	What It Means
Network namespace	Each container has its own network stack including its own localhost.
Bridge network	A virtual network that allows containers to communicate with each other.
Container IP	Each container gets an IP address on the Docker network (e.g., 172.18.0.x).
Service-name resolution	Containers can reach each other using container names (built-in DNS).
localhost in a container	Refers only to that container (127.0.0.1).
Host gateway	The bridge gateway (e.g., 172.18.0.1) or host.docker.internal on Docker Desktop.

8. COMMON COMMANDS

Purpose	Command
List containers	`docker ps`
Inspect network	`docker network inspect demo-net`
Exec into container	`docker exec -it api sh`
Check container IP	`ip addr show`
Test connectivity	`curl -I http://web`
Check routing	`ip route`
Check DNS resolution	`cat /etc/resolv.conf`
View network interfaces	`ip a`

9. TROUBLESHOOTING STEPS

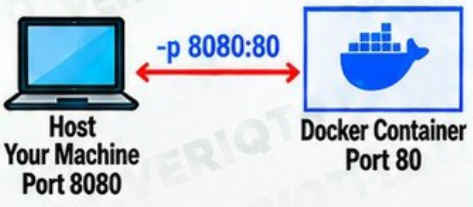
1. Confirm both containers are on the same network.
2. Verify container names and IP addresses (docker network inspect).
3. Check if the service is listening inside the container (ss -lntp).
4. Test connectivity using container name and IP (curl).
5. Remember that localhost is container-specific.
6. Use the correct host address if you need to reach the host.
7. Check firewall rules or network policies if connectivity fails.

VERIQTA

Your Container Runs, but the Published Port Does Not Work

Investigate -p 8080:80, port publishing, container listening addresses, and host-to-container traffic. Diagnose an application bound to the wrong interface.

1. HOW PORT PUBLISHING WORKS



- The -p 8080:80 option maps host port 8080 to container port 80.
- Traffic to http://<host-ip>:8080 is forwarded to port 80 inside the container.
- The container must be listening on the correct interface for this to work.

2. RUN A CONTAINER WITH A PUBLISHED PORT

```
docker run -d -p 8080:80 --name web nginx
```

```
# Check running containers
docker ps
CONTAINER ID   IMAGE   PORTS                  NAMES
a1b2c3d4e5f6   nginx   0.0.0.0:8080->80/tcp   web

# Check the port mapping
docker port web
80/tcp -> 0.0.0.0:8080
```

- The container is running and port 8080 on the host is mapped to port 80 in the container.
- docker port confirms the actual mapping.

3. TEST ACCESS FROM THE HOST

```
curl -v http://localhost:8080
```

```
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080
> GET / HTTP/1.1
> Host: localhost:8080
< HTTP/1.1 200 OK
< Server: nginx/1.25.3
< Content-Type: text/html
...
```

- A successful response confirms that host-to-container traffic is working.
- If you get connection refused or timeout, there is a configuration or networking issue.

4. CHECK WHAT THE APPLICATION IS LISTENING ON

```
docker exec -it web ss -lntp
```

```
State   Recv-Q  Send-Q  Local Address:Port   Process
LISTEN  0       511     0.0.0.0:80           nginx
LISTEN  0       511     [::]:80              nginx
```

- The application must listen on 0.0.0.0 (all interfaces) inside the container.
- If it is listening only on 127.0.0.1, it will not be reachable from the host even with -p.

5. EXAMPLE: APPLICATION BOUND TO THE WRONG INTERFACE

```
docker run -d -p 8080:8000
--name app python:3
python -m http.server 8000 --bind 127.0.0.1
```

```
docker exec -it app ss -lntp
State   Recv-Q  Send-Q  Local Address:Port
LISTEN  0       5       127.0.0.1:8000
```

- The application is bound to 127.0.0.1 inside the container.
- This is only accessible from inside the container, not from the host.
- Even though -p 8080:8000 is set, the port will not work.

6. FIX: BIND TO ALL INTERFACES

```
docker run -d -p 8080:8000
--name app python:3
python -m http.server 8000
--bind 0.0.0.0
```

```
docker exec -it app ss -lntp
State   Recv-Q  Send-Q  Local Address:Port
LISTEN  0       5       0.0.0.0:8000
```

- Now the application listens on 0.0.0.0 inside the container.
- The host can access it via http://localhost:8080.

7. USE LOGS TO INVESTIGATE

```
docker logs app
```

```
Serving HTTP on 0.0.0.0 port 8000 ...
127.0.0.1 - - [28/Oct/2026 10:15:23] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [28/Oct/2026 10:15:30] "GET / HTTP/1.1" 200 -
```

- Logs show which client IP is connecting to the container.
- If you see no requests, the traffic is not reaching the application.
- Use docker logs to confirm the application is running and receiving connections.

8. COMMON PORT PUBLISHING ISSUES

Issue	Symptom	Likely Cause
Connection refused	Cannot connect to host port	Application not running or bound to 127.0.0.1 inside container
Timeout	No response	Firewall, wrong host IP, or container not running
Works inside container but not from host	curl works in container but fails on host	Application bound to 127.0.0.1 instead of 0.0.0.0
Wrong port mapping	Different port than expected	Incorrect -p mapping or port already in use
Container running but no logs	No requests in logs	Traffic not reaching container or wrong network settings

9. TROUBLESHOOTING STEPS

1. Verify the container is running (docker ps).
2. Check the port mapping (docker port).
3. Confirm the application is listening on 0.0.0.0 inside the container.
4. Test access from the host (curl).
5. Check container logs (docker logs).
6. Ensure no firewall is blocking the port.
7. Verify the correct host IP and port.
8. Test with a simple image like nginx to rule out application issues.

10. JUNIOR ENGINEER TIPS

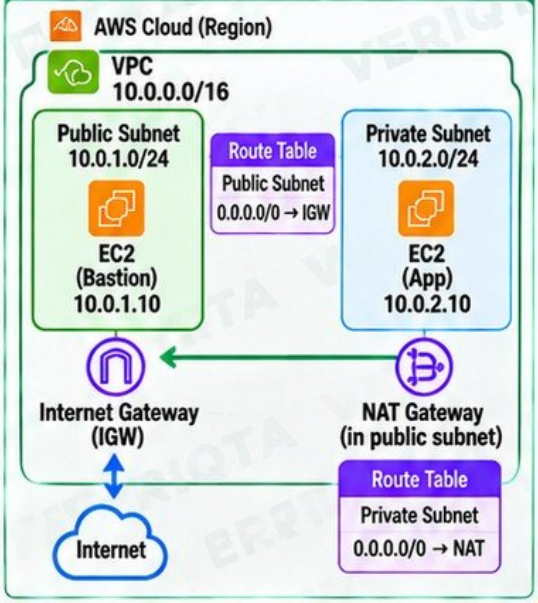
- Always check what address the application is listening on.
- Use docker port to confirm the actual host-to-container mapping.
- Test with a simple image (nginx or http.server) before troubleshooting a complex application.
- Remember that localhost inside a container is the container itself, not the host.
- Check firewall rules if the port is still not reachable.

VERIQTA

Design a Small AWS VPC and Trace a Failed Connection

Connect CIDR planning to subnets, route tables, internet gateways, NAT gateways, security groups, and network ACLs. Diagnose why an instance in a private subnet cannot reach an external endpoint.

1. VPC ARCHITECTURE OVERVIEW



2. CHECK VPC AND SUBNETS

```
aws ec2 describe-vpcs --query
'Vpcs[*].[VpcId,CidrBlock]'
```

VpcId	CidrBlock
vpc-0a1b2c3d4e	10.0.0.0/16

```
aws ec2 describe-subnets --filters
"Name=vpc-id,Values=vpc-0a1b2c3d4e" \
--query 'Subnets[*].[SubnetId,CidrBlock,
MapPublicIpOnLaunch]'
```

SubnetId	CidrBlock	MapPublicIpOnLaunch
subnet-1111	10.0.1.0/24	true
subnet-2222	10.0.2.0/24	false

- VPC provides isolated network space.
- Public subnet has a route to the Internet Gateway (IGW).
- Private subnet does not have a direct route to the internet.

3. INSPECT ROUTE TABLES

```
aws ec2 describe-route-tables
--filters "Name=vpc-id,Values=vpc-0a1b2c3d4e"
```

Public Subnet Route Table

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-0a1b2c3d4e

Private Subnet Route Table

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	nat-0a1b2c3d4e

- Public subnet uses an Internet Gateway for external access.
- Private subnet uses a NAT Gateway.
- If the private subnet route is missing or wrong, instances cannot reach the internet.

4. CHECK SECURITY GROUPS

```
aws ec2 describe-security-groups
--group-ids sg-0abc1234
```

Inbound rules (example)

Type	Protocol	Port	Source
SSH	TCP	22	0.0.0.0/0
HTTP	TCP	80	0.0.0.0/0

- Security groups are stateful firewalls.
- They control inbound and outbound traffic for instances.
- Outbound traffic is allowed by default unless explicitly restricted.

5. CHECK NETWORK ACLS

```
aws ec2 describe-network-acls
--filters "Name=vpc-id,Values=vpc-0a1b2c3d4e"
```

Rule #	Type	Protocol	Port	Action	CIDR
100	All	All	All	ALLOW	0.0.0.0/0
*	All	All	All	DENY	0.0.0.0/0

- Network ACLs are stateless.
- Both inbound and outbound rules must allow traffic.
- If return traffic is blocked, connections will fail even if the security group allows it.

6. TEST CONNECTIVITY FROM PRIVATE INSTANCE

```
# Try to reach an external endpoint
curl -v https://example.com
*   Trying 93.184.216.34:443...
*  connect to 93.184.216.34 port 443
   failed: Connection timed out
curl: (28) Connection timed out
```

- A timeout usually means the instance cannot reach the internet.
- Check the NAT Gateway, route table, NACL and security group outbound rules.
- Compare with the bastion in the public subnet to confirm internet access works there.

7. COMMON FAILURE SCENARIOS

Scenario	Symptom	Likely Cause
No internet from private instance	Timeout	No NAT Gateway or missing route
Can reach internet but no response	Connection timed out	NACL blocking return traffic
Bastion works, private does not	Bastion has IGW, private uses NAT	NAT Gateway misconfigured or in wrong subnet
Connection refused	Immediate refusal	Security group or NACL denying traffic
DNS fails	Could not resolve host	No DNS resolution (VPC DNS settings or outbound blocked)

8. COST AWARENESS

Component	Cost Considerations
NAT Gateway	Charged per hour and per GB of data processed.
Data Transfer	Outbound internet data incurs additional charges.
Elastic IP	Charged if idle (not attached to a running resource).
VPC Endpoints	Can reduce internet data costs for AWS services.

- Use VPC endpoints for AWS services (S3, ECR, etc.) to avoid NAT costs.
- Stop unused resources to prevent charges.

9. TROUBLESHOOTING STEPS

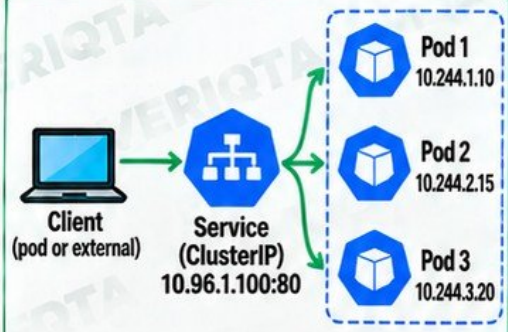
1. Verify the private instance cannot reach the internet (curl).
2. Check the instance's route table.
3. Verify the NAT Gateway is in a public subnet and has an Elastic IP.
4. Check NACL inbound and outbound rules (including return traffic).
5. Confirm security group outbound rules allow the traffic.
6. Test DNS resolution (dig or nslookup).
7. Fix the identified issue and test again.

VERIQTA

A Kubernetes Pod Is Healthy, but Its Service Cannot Reach It

Trace Pod IPs, ClusterIP Services, selectors, EndpointSlices, and target ports. Investigate a Service that has no matching ready backends.

1. ARCHITECTURE OVERVIEW



- A Service selects Pods using labels (selectors).
- It routes traffic to healthy, ready Pods.
- If no Pods match the selector or are not ready, the Service has no backends and cannot reach the application.

2. CHECK PODS ARE RUNNING AND READY

```
kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	IP
app-7c6d9f5b7-abc12	1/1	Running	0	10.244.1.10
app-7c6d9f5b7-def34	1/1	Running	0	10.244.2.15
app-7c6d9f5b7-ghi56	1/1	Running	0	10.244.3.20

- Pods show READY 1/1 and STATUS Running so the application is healthy.
- Each Pod has its own cluster IP (e.g. 10.244.x.x) inside the cluster network.

3. CHECK THE SERVICE

```
kubectl get svc
```

NAME	TYPE	CLUSTER-IP	PORT(S)
kubernetes	ClusterIP	10.96.0.1	443/TCP
app-service	ClusterIP	10.96.1.100	80/TCP

- The Service has a ClusterIP (10.96.1.100) and port 80.
- Clients inside the cluster should be able to reach it using the Service IP or DNS name (app-service).
- If it is not working, check the selector, endpoints and target port.

4. CHECK THE SERVICE SELECTOR

```
kubectl describe svc app-service
```

```
Name:          app-service
Namespace:     default
Type:          ClusterIP
IP:            10.96.1.100
Port:          80/TCP
TargetPort:    8080/TCP
Selector:      app: myapp
Endpoints:     <none>
```

- The selector (app: myapp) must match the labels on the Pods.
- If selectors do not match, the Service will have no endpoints (<none>).
- TargetPort is the port on the Pod where the app is listening (e.g. 8080).

5. CHECK ENDPOINTSLICES

```
kubectl get endpointslices
```

NAME	ADDRESSES	PORTS	READY
app-service-abc12	10.244.1.10	8080	true
app-service-def34	10.244.2.15	8080	true
app-service-ghi56	10.244.3.20	8080	true

- EndpointSlices show the actual Pod IPs and ports that the Service will send traffic to.
- If there are no EndpointSlices or the list is empty, the Service has no ready backends.
- The READY column must be true.

6. COMMON REASONS FOR NO ENDPOINTS

Reason	Details
Selector mismatch	Service selector labels do not match Pod labels.
Pods not Ready	Readiness probe failing or Pod still starting.
Wrong namespace	Service and Pods are in different namespaces.
Wrong target port	Service targetPort does not match the container port.
Pods have no matching labels	Label key or value is incorrect.
Pods failing or CrashLooping	Pods exist but are not ready.

7. VERIFY POD LABELS

```
kubectl get pods --show-labels
```

NAME	READY	STATUS	LABELS
app-7c6d9f5b7-abc12	1/1	Running	app=myapp
app-7c6d9f5b7-def34	1/1	Running	app=myapp
app-7c6d9f5b7-ghi56	1/1	Running	app=myapp

- Pods must have the labels that match the Service selector.
- If the labels are different (e.g. app=web), update the Service selector or the Pod labels.

8. TEST CONNECTIVITY INSIDE THE CLUSTER

```
# From another pod in the same namespace
kubectl run -it --rm debug --image=busybox -- sh
/ # nslookup app-service
Server:    10.96.0.10
Address 1: 10.96.0.10 kube-dns.kube-system.svc
/ # wget -O- http://app-service:80
Connecting to app-service (10.96.1.100:80)
<!DOCTYPE html>
<html>App is working</html>
```

- Use a temporary pod to test the Service DNS name.
- If DNS resolves but the request fails, check endpoints and target port.
- If DNS does not resolve, check CoreDNS and namespace.

9. TROUBLESHOOTING STEPS

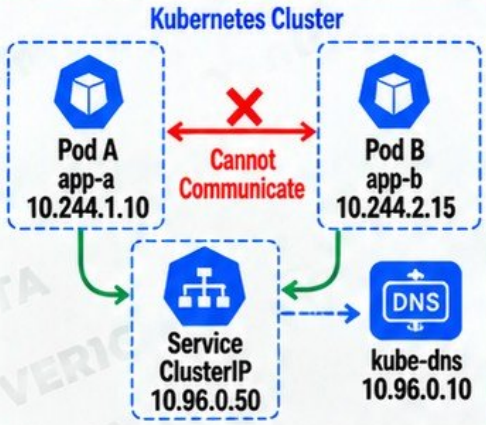
1. Check that Pods are running and ready (kubectl get pods).
2. Verify Pod labels match the Service selector.
3. Check that the Service has endpoints (kubectl get endpointslices).
4. Confirm the targetPort matches the container port.
5. Test connectivity using a temporary pod (curl).
6. Check readiness probes and Pod logs if Pods are not ready.
7. Ensure you are using the correct namespace.
8. After making changes, test again.

VERIQTA

Two Kubernetes Pods Cannot Communicate: Is It DNS or NetworkPolicy?

Diagnose Pod-to-Pod and Pod-to-Service connectivity. Check DNS, ports, selectors, and NetworkPolicy rules before changing cluster networking.

1. SCENARIO OVERVIEW



- Pods are running but cannot reach each other or a Service.
- The issue can be DNS, port, selector, or NetworkPolicy.
- Use a systematic approach to identify the root cause.

2. TEST POD-TO-POD CONNECTIVITY

```
kubectl exec -it pod-a -- curl -v http://10.244.2.15:8080
```

```
*   Trying 10.244.2.15:8080...
*  connect() timed out after 5001 ms
curl: (28) Connection timed out
```

- Test direct Pod IP connectivity using curl or nc.
- If it times out, check NetworkPolicy, security groups (CNI), or if the target Pod is listening on the expected port.
- If connection is refused, the Pod is reachable but the application is not listening on that port.

3. CHECK DNS RESOLUTION

```
kubectl exec -it pod-a -- nslookup app-b
```

```
Server:     10.96.0.10
Address:    10.96.0.10#53

Name:    app-b.default.svc.cluster.local
Address: 10.96.0.50
```

- Use nslookup or dig to verify DNS resolution.
- If DNS fails, check CoreDNS, the Service name, and the namespace.
- Use the full service name (app-b.default.svc.cluster.local) or the short name (app-b) inside the same namespace.

4. CHECK THE SERVICE AND SELECTORS

```
kubectl get svc app-b -o wide
```

NAME	TYPE	CLUSTER-IP	PORT(S)	SELECTOR
app-b	ClusterIP	10.96.0.50	8080/TCP	app=app-b

```
kubectl get pods --show-labels | grep app-b
```

```
app-b-7c6d9f5b7-abc12   1/1   Running   app=app-b
```

- The Service routes traffic to Pods that match the selector.
- Ensure the Pod labels match the Service selector.
- Verify the Service port and targetPort.

5. CHECK NETWORKPOLICY RULES

```
kubectl get networkpolicy -n default
```

NAME	POD-SELECTOR	AGE
deny-all	<none>	5d
allow-app	app=app-b	2d

```
kubectl describe networkpolicy allow-app
```

```
Ingress:
  From:
    - podSelector:
        matchLabels:
          app: app-a
  Ports:
    - protocol: TCP
      port: 8080
```

- NetworkPolicy can block Pod-to-Pod traffic.
- Check both ingress and egress rules.
- Ensure there is a rule allowing traffic from Pod A to Pod B on the correct port.

6. VERIFY THE TARGET PORT

```
kubectl exec -it pod-b -- ss -lntp
```

State	Recv-Q	Send-Q	Local Address:Port	Process
LISTEN	0	128	0.0.0.0:8080	nginx
LISTEN	0	128	[::]:8080	nginx

- Confirm the application inside the Pod is listening on the expected port.
- If it is only listening on 127.0.0.1, it will not be reachable from other Pods.
- Ensure the Service targetPort matches the container port.

7. TEST POD-TO-SERVICE CONNECTIVITY

```
kubectl exec -it pod-a -- curl -v http://app-b:8080
```

```
> GET / HTTP/1.1
> Host: app-b:8080
< HTTP/1.1 200 OK
< Server: nginx/1.25.3
< Content-Type: text/html
...
```

- Test using the Service name (app-b) and port.
- A successful response confirms DNS and networking are working.
- If it fails, recheck DNS, selectors, NetworkPolicy, and the target port.

8. COMMON ROOT CAUSES

Issue	Symptom
DNS resolution failure	nslookup fails or returns no IP address
Service selector mismatch	Service has no endpoints
NetworkPolicy blocking traffic	Connection times out
Wrong targetPort	Connection refused
Application bound to localhost	Pod not reachable from other Pods
Wrong namespace	Cannot resolve short name
CNI or cluster network issue	Intermittent or complete connectivity failure

9. TROUBLESHOOTING STEPS

1. Verify both Pods are Running and Ready.
2. Check DNS resolution (nslookup or dig).
3. Confirm Service selectors match Pod labels.
4. Check the Service port and targetPort.
5. Inspect NetworkPolicy rules (ingress and egress).
6. Verify the application is listening on the correct interface and port.
7. Test Pod-to-Pod and Pod-to-Service connectivity using curl.
8. Make one change at a time and retest.

VERIQTA

The Website Is Slow, but CPU and Memory Look Normal

Measure DNS lookup time, connection time, TLS negotiation, and time to first byte. Distinguish network latency from slow upstream application processing.

1. UNDERSTAND THE REQUEST TIMELINE



DNS lookup time	Connection time	TLS handshake time	TTFB (server processing)

- Each step adds to the total response time.
- A slow website can be caused by delays at any of these layers.
- We need to measure each step to find the real bottleneck.

2. MEASURE DETAILED TIMINGS WITH CURL

```
curl -o /dev/null -s -w "
    DNS Lookup:     %{time_namelookup}s
    Connect Time:   %{time_connect}s
    TLS Handshake:  %{time_appconnect}s
    Time to First Byte: %{time_starttransfer}s
    Total Time:     %{time_total}s
" https://example.com
```

```
DNS Lookup:          0.023358s
Connect Time:        0.045123s
TLS Handshake:       0.112634s
Time to First Byte: 1.243567s
Total Time:          1.426892s
```

- time_namelookup: DNS resolution time.
- time_connect: TCP connection time.
- time_appconnect: TLS negotiation time (HTTPS).
- time_starttransfer: Time to first byte (TTFB).
- time_total: Total time for the request.

3. CHECK NETWORK LATENCY WITH PING

```
ping example.com
```

```
PING example.com (93.184.216.34) 56(84) bytes of data.
64 bytes from 93.184.216.34: icmp_seq=1 ttl=52 time=22.4 ms
64 bytes from 93.184.216.34: icmp_seq=2 ttl=52 time=21.9 ms
64 bytes from 93.184.216.34: icmp_seq=3 ttl=52 time=22.1 ms
--- example.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss
rtt min/avg/max/mdev = 21.912/22.133/22.421/0.208 ms
```

- Ping shows basic network latency and packet loss.
- High latency or packet loss can make the website feel slow.
- If ping is normal, the issue may be DNS, TLS, or the application.

4. TRACE THE NETWORK PATH WITH MTR

```
mtr -rwzbc 10 example.com
```

HOST	Loss%	Snt	Last	Avg	Best	Wrst	StDev
1. 192.168.1.1	0.0%	10	0.5	0.6	0.4	1.2	0.2
2. 10.0.0.1	0.0%	10	5.1	5.3	5.0	6.1	0.3
3. 172.16.0.1	0.0%	10	12.4	12.8	12.0	15.1	0.9
4. 93.184.216.34	0.0%	10	22.1	22.3	21.8	23.6	0.5

- mtr combines ping and traceroute.
- It shows each hop, latency, and packet loss.
- High loss or high latency on a specific hop indicates a network issue between you and the destination.

5. COMPARE HTTP vs HTTPS

```
curl -o /dev/null -s -w "Total Time: %{time_total}s\n" http://example.com
```

```
curl -o /dev/null -s -w "Total Time: %{time_total}s\n" https://example.com
```

```
HTTP   Total Time: 0.342561s
HTTPS  Total Time: 1.426892s
```

- If HTTPS is much slower, the issue may be TLS negotiation (e.g., slow certificate chain or OCSP/CRL checks).
- If both are slow, the issue may be network latency or slow application processing.

6. IDENTIFY SLOW UPSTREAM PROCESSING

```
curl -w "  nDNS: %{time_namelookup}s
           Connect: %{time_connect}s
           TLS:   %{time_appconnect}s
           TTFB:  %{time_starttransfer}sn
           Total: %{time_total}s
           -o /dev/null -s https://example.com
```

```
DNS:      0.023358s
Connect:  0.045123s
TLS:      0.112634s
TTFB:     2.345678s  ← High!
Total:    2.589123s
```

- If DNS, connect, and TLS are fast but TTFB is high, the application is slow to respond.
- Check application logs, database queries, or upstream services.

7. TROUBLESHOOTING STEPS

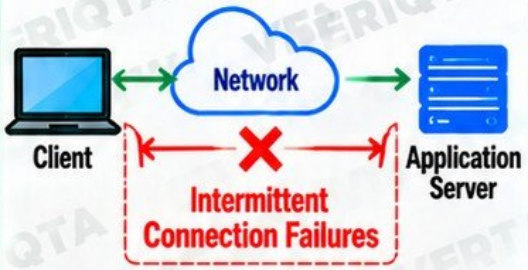
1. Check overall response time (curl -w).
2. Verify basic network latency (ping).
3. Trace the path for packet loss (mtr).
4. Compare HTTP vs HTTPS timings.
5. Identify which step is slow (DNS, connect, TLS, or TTFB).
6. If TTFB is high, check application and upstream services.
7. Check for external dependencies (e.g., APIs, databases).
8. Make one change at a time and test again.

VERIQTA

Intermittent Connection Failures: Investigate Before Restarting

Examine connection states, retransmissions, socket errors, connection pressure, and possible ephemeral port exhaustion. Learn why restarting a service can hide evidence.

1. UNDERSTAND THE SYMPTOM



- Connections work sometimes and fail at other times.
- Failures can be caused by network issues, server resource limits, firewall rules, or ephemeral port exhaustion.
- Do not restart the service immediately. Gather evidence first.

2. CHECK CONNECTION STATES

```
ss -s
```

```
Total: 2487
TCP:   2360 (estab 1420, closed 910,
       orphaned 12, timewait 805)

Transport  Total     IP        IPv6
* 2360     2487      0         0

TCP:
  1420 established
  12   orphaned
  805  timewait
  42   synrecv
  5    synsent
```

- Look at the number of established, timewait, synsent, and synrecv connections.
- A very high number of timewait or synsent connections can indicate connection pressure or external issues.

3. INSPECT ACTIVE CONNECTIONS

```
ss -tan | head -10
```

State	Recv-Q	Send-Q	Local Address:Port
ESTAB	0	0	10.0.0.10:22
ESTAB	0	0	10.0.0.10:443
TIME-WAIT	0	0	10.0.0.10:50234
SYN-SENT	0	1	10.0.0.10:49120
ESTAB	0	0	10.0.0.10:54321

- Check for many connections in SYN-SENT or TIME-WAIT state.
- Look for unusual remote addresses or ports.
- Identify if connections are stuck in a specific state.

4. CHECK RETRANSMISSIONS AND SOCKET ERRORS

```
nstat | egrep "RetransSegs|OutRsts|InErrs|OutErrors"
```

```
TcpRetransSegs                  12543
TcpOutRsts                      342
IpInErrors                      28
IpOutErrors                     17
TcpExtListenOverflows           5
TcpExtSyncookiesSent            0
```

- High TcpRetransSegs indicates packet loss or network congestion.
- TcpOutRsts can indicate that connections are being reset.
- IP errors suggest network or interface problems.

5. CHECK EPHEMERAL PORT RANGE

```
cat /proc/sys/net/ipv4/ip_local_port_range
```

```
32768   60999
```

- This range is used for outgoing client connections.
- If many connections are created and closed quickly, ports can remain in TIME-WAIT and exhaust the range.
- When ports are exhausted, new connections will fail with "Cannot assign requested address".

6. CHECK TIME-WAIT CONNECTIONS

```
ss -tan state time-wait | wc -l
```

```
805
```

- A large number of TIME-WAIT connections is normal under high traffic but can cause issues if the ephemeral port range is too small.
- If the number keeps growing, the system may run out of ephemeral ports.

7. EXAMPLE: EPHEMERAL PORT EXHAUSTION

```
curl -v http://example.com
```

```
*   Trying 93.184.216.34:80...
*   connect() failed: Cannot assign
    requested address
curl: (7) Failed to connect to example.com
port 80: Cannot assign requested address
```

- This error means the system has run out of ephemeral ports.
- Check TIME-WAIT connections and the port range.
- Increase the port range or reduce the connection rate if needed.

8. COMMON CAUSES AND SIGNS

Cause	Signs / Evidence
Network packet loss	High retransmissions, connection timeouts
Server under load	Slow responses, connection queue buildup
Ephemeral port exhaustion	Many TIME-WAIT, cannot assign requested address
Firewall or security rules	SYN-SENT connections, timeouts
Remote server issues	Intermittent timeouts, high latency
Application bug	Unclosed connections, connection leaks

9. TROUBLESHOOTING STEPS

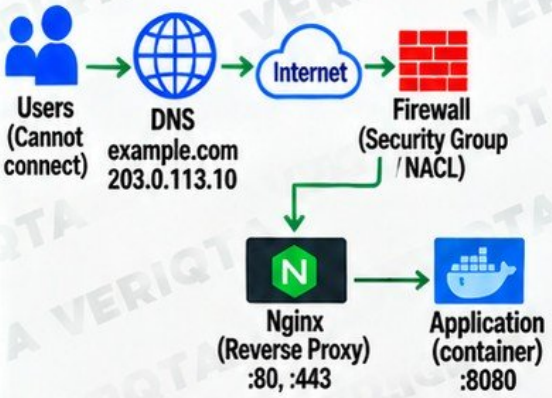
1. Check connection states (ss -s, ss -tan).
2. Look for retransmissions and errors (nstat).
3. Check TIME-WAIT count.
4. Verify the ephemeral port range.
5. Test connectivity to the remote host (curl, ping).
6. Check firewall rules and network path.
7. Review application logs for connection leaks.
8. Only restart the service after you have gathered and saved evidence.

VERIQTA

Final Production Lab: The Deployment Succeeded, but Customers Cannot Connect

Investigate a simulated incident involving an application, Nginx, DNS, TLS, and network filtering. Work through evidence collection, fault isolation, safe remediation, and verification.

1. SCENARIO OVERVIEW



- A new version of the application was deployed.
- Deployment completed successfully.
- However, customers cannot connect to the website.
- Your task is to investigate, find the root cause, and restore connectivity without causing further issues.

2. VERIFY DNS RESOLUTION

```
dig example.com +short
```

```
203.0.113.10
```

```
nslookup example.com
```

```
Server:    8.8.8.8
Address:   8.8.8.8#53

Name:      example.com
Address:   203.0.113.10
```

- Confirm the domain resolves to the correct IP address.
- Check for DNS propagation issues if multiple records exist.
- Compare results from different networks or DNS servers.

3. TEST EXTERNAL CONNECTIVITY

```
curl -v https://example.com
```

```
*   Trying 203.0.113.10:443...
*  connect() timed out after
   10002 millisecends
curl: (28) Connection timed out
```

```
openssl s_client -connect
example.com:443
```

```
connect: Connection timed out
```

- Test both HTTP (port 80) and HTTPS (port 443).
- Identify if the connection is refused, timed out, or returning an error.
- Determine if the issue is network filtering, Nginx, or the application.

4. CHECK NETWORK FILTERING

```
sudo nft list ruleset | grep -E '80|443'
```

```
chain input {
    type filter hook input priority 0;
    tcp dport { 80, 443 } counter accept
}
```

- Verify firewall rules (nftables/iptables).
- Check cloud security groups and network ACLs (in AWS or your environment).
- Ensure ports 80 and 443 are allowed from the internet.
- Use counters to confirm if traffic is reaching the server.

5. VERIFY NGINX STATUS AND CONFIG

```
sudo systemctl status nginx
```

```
● nginx.service - A high performance
  web server
     Active: active (running)
  Main PID:  1234 (nginx)
```

```
sudo nginx -t
```

```
nginx: the configuration file
/etc/nginx/nginx.conf syntax is ok
nginx: configuration file test is
successful
```

- Confirm Nginx is running and healthy.
- Check for configuration errors.
- Verify that Nginx is listening on 80/443 and proxying to the correct backend.

6. CHECK APPLICATION HEALTH

```
docker ps
```

CONTAINER ID	IMAGE	STATUS	PORTS
a1b2c3d4	app:latest	Up 10m	8080/tcp

```
curl http://127.0.0.1:8080/health
```

```
OK
```

- Confirm the application container is running and healthy.
- Test the application directly on its port.
- Ensure the application is listening on the correct interface (0.0.0.0 not 127.0.0.1).

7. CHECK TLS CERTIFICATE (IF HTTPS)

```
openssl s_client -connect example.com:443
 -servername example.com
```

```
Certificate chain
 0 s:CN = example.com
   i:CN = Let's Encrypt R3
   Verify return code: 0 (ok)
```

- Verify the TLS certificate is valid and not expired.
- Check that the correct certificate is being served.
- Ensure the domain name matches the certificate (SNI).
- Identify any TLS handshake or cipher issues.

8. BUILD THE INCIDENT TIMELINE

Time (UTC)	Event	Observation
10:00	Deployment completed	New version deployed successfully
10:05	Customer reports website down	Cannot connect (timeout)
10:10	DNS check	Correct IP address
10:15	Firewall check	Traffic blocked on 443
10:20	Nginx check	Running and healthy
10:25	Fix applied	Opened port 443
10:30	Verification	Website accessible

- Document what you found, the evidence, changes made, and the final outcome.

9. ROOT CAUSE AND REMEDIATION

Root Cause (example)

Port 443 was blocked in the firewall rules after the deployment, preventing external access to Nginx.

Remediation

- ✓ Updated firewall rule to allow port 443
- ✓ Verified Nginx and application health
- ✓ Confirmed external connectivity
- ✓ Monitored for any further issues

Final Verification

- ✓ Website accessible from multiple networks
- ✓ HTTPS certificate valid
- ✓ Application responding normally